



파이썬 프로그래밍

9~10장: 예외 처리와 파일

Fall Semester 2025-02



Outline

1. 예외 처리 구현
2. try-except 문법
3. 파일
4. 파일 입출력
5. with문

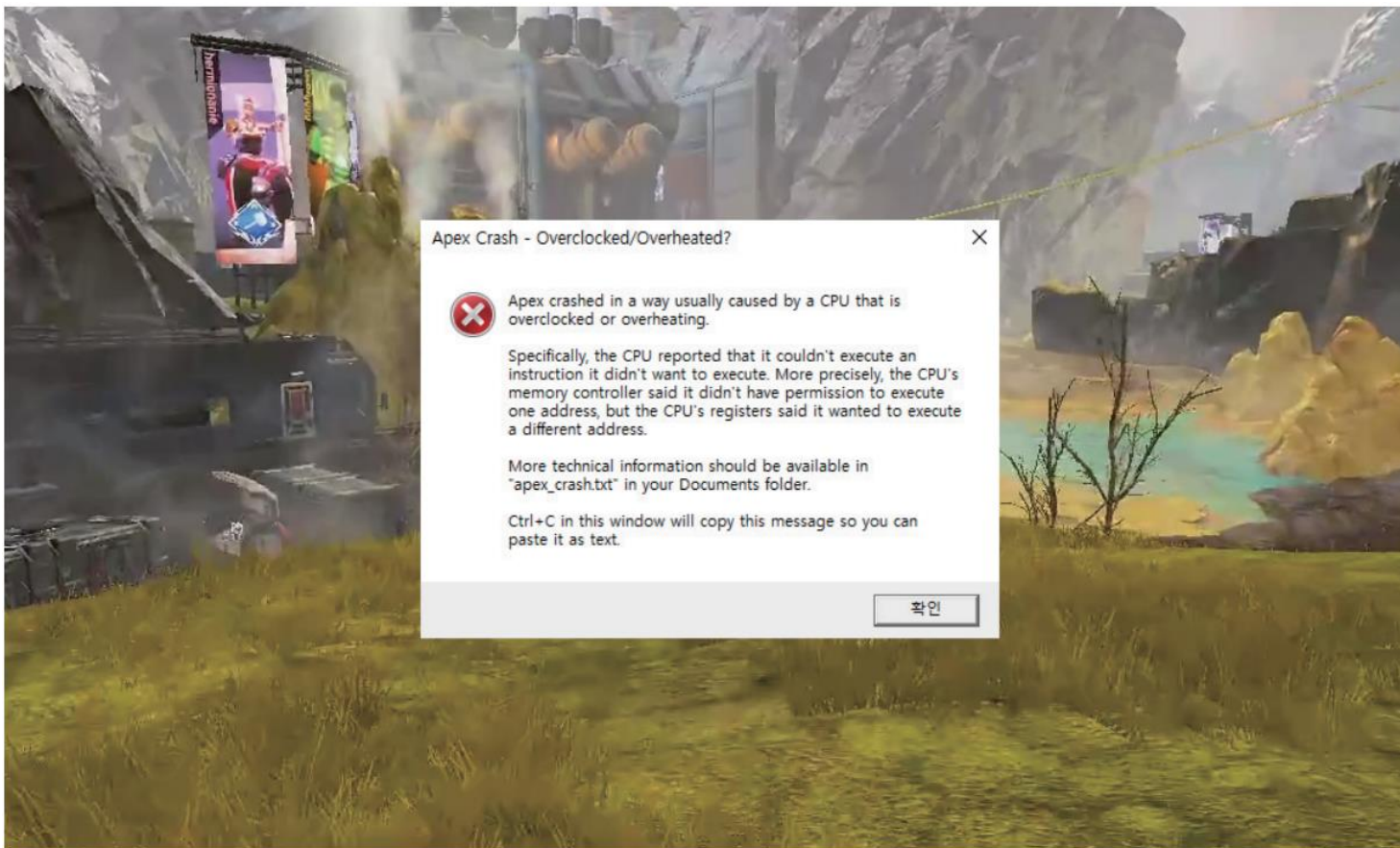


9.1 안전한 코딩을 위한 예외 처리



오류

- 특정한 언어에서 정해진 문법을 따르지 않는 명령이 입력되어 프로그램이 문제를 일으키는 것



“에러”는 크게 두 가지로 나눌 수 있음:

- 1) **구문 오류(Syntax Error)**
→ 프로그램이 실행되기 전에(컴파일 단계에서) 발견되는 오류
- 2) **예외(Exception)** → 프로그램 실행 중(runtime)에 발생하는 오류



9.2 try-except 문의 문법



- 예외처리 **exception handling**를 위해 사용
- try 절 - 예외가 발생할 우려(concern)가 있는 코드를 입력
- except 절 - 오류가 발생했을 때 처리할 내용을 담음
- 예외가 발생하지 않으면 except를 건너뛰고 예외가 발생하면 오류를 확인하여 except의 매칭되는 부분으로 넘겨준다.

try:

{예외가 발생할 우려가 있는 코드}

except [예외의 타입]:

{예외가 발생할 경우 실행되는 코드}



파이썬의 예외

BaseException	ProcessLookupError
Exception	TimeoutError
ArithmeticError	ReferenceError
FloatingPointError	RuntimeError
OverflowError	NotImplementedError
ZeroDivisionError	RecursionError
AssertionError	StopIteration
AttributeError	StopAsyncIteration
BufferError	SyntaxError
EOFError	IndentationError
ImportError	TabError
ModuleNotFoundError	SystemError
LookupError	TypeError
IndexError	ValueError
KeyError	UnicodeError
MemoryError	UnicodeDecodeError
NameError	UnicodeEncodeError
UnboundLocalError	UnicodeTranslateError
OSError	Warning
BlockingIOError	BytesWarning
ChildProcessError	DeprecationWarning
ConnectionError	FutureWarning
BrokenPipeError	ImportWarning
ConnectionAbortedError	PendingDeprecationWarning
ConnectionRefusedError	ResourceWarning
ConnectionResetError	RuntimeWarning
FileExistsError	SyntaxWarning
FileNotFoundError	UnicodeWarning
InterruptedError	UserWarning
IsADirectoryError	GeneratorExit
NotADirectoryError	KeyboardInterrupt
PermissionError	SystemExit

예외의 종류가 매우 많기 때문에 이 예외의 목록을 만들어 두고 해당하는 예외가 발생하면 번역기가 “except 예외” 문에서 처리하도록 제어를 이동함

<https://docs.python.org/3/library/exceptions.html>



9.3 구체적인 예외를 명시하기 / Specify specific exceptions



```
try:
    b = 2 / 0
    a = 1 + 'hundred'
except ZeroDivisionError:
    print('0으로 나누는 오류')
except TypeError:
    print('지원되지 않은 연산자를 사용하는 오류')
```

TypeError: Raised when an operation or function is applied to an object of inappropriate type. The associated value is a string giving details about the type mismatch.

실행결과

0으로 나누는 오류



9.3 구체적인 예외를 명시하기



- `except XXX :`와 함께 `except Exception as e:`를 통해 보편적인 예외처리 기능을 넣어 주는 것이 더 안전한 코딩이 된다.

```
try:
    # b = 2 / 0
    a = 1 + 'hundred'
except ZeroDivisionError: # 0으로 나누는 예외에 대한 처리
    print('0으로 나누는 오류')
except TypeError:        # 지원되지 않는 연산자 사용 예외에 대한 처리
    print('지원되지 않은 연산자를 사용하는 오류')
except Exception as e:   # 일반적인 예외 처리
    print('error :', e)
    print('나눗셈과 연산자를 다시 확인하세요')
```



9.4 try-except-else 문



- 예외가 발생하지 않을 경우 else문을 사용하여 그 결과값을 출력해 보기
- 예외가 발생하지 않은 경우에 실행되므로 반드시 except절 바로 다음에 위치해야 함

```
try:
    a, b = input('두 수를 입력하시오: ').split()
    result = int(a) / int(b)
except ZeroDivisionError:
    print('오류 : 0으로 나눴습니다.')
except ValueError:
    print('오류 : 입력 값이 정수나 실수가 아닙니다.')
except:
    print('오류 : 10 2와 같이 두 정수를 입력하세요.')
else:    # 예외가 발생하지 않은 경우에 실행
    print('{} / {} = {}'.format(a, b, result))
```


예외 처리로 pass문 사용

- 실행 오류가 발생했을 때, 오류를 처리하지 않고 프로그램 실행을 지속시키기 위해 pass문을 사용하는 경우가 있음

```
try:
    print(3 / 0)    # 오류 발생
except ZeroDivisionError:
    pass           # 예외 처리로 아무것도 안함
print("오류가 발생함에도 여기까지 출력됨")
```



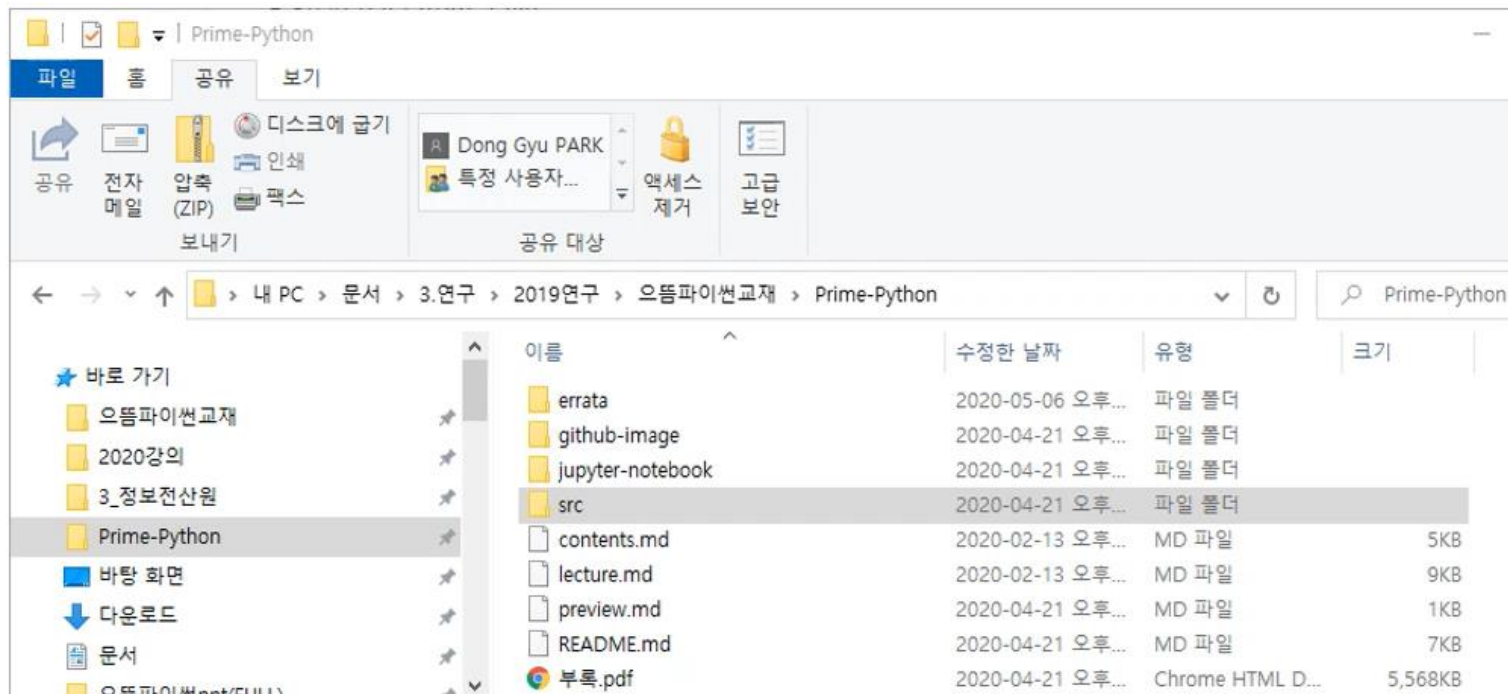
9.7 파일 입출력



9.7.1 파일이란 무엇인가

• **파일** file : 파일 확장자이다. 예시: .txt, .png, .jpg, 등

- 컴퓨터의 저장 장치 내에 데이터를 저장하기 위해 사용하는 논리적인 단위
- **하드 디스크** hard disk나 **외장 디스크** external disk와 같은 저장 장치에 저장한 후 필요할 때 다시 불러서 사용하는 것이 가능





9.7 파일 입출력



파이썬에서 파일을 사용하기 위한 절차

1. 파일 열기

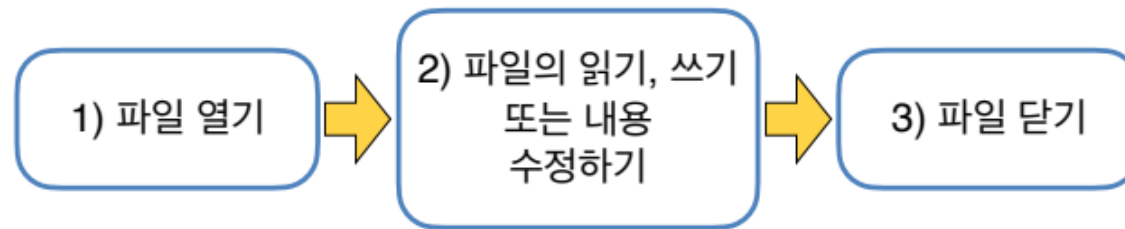
- 저장 장치 내의 위치와 파일 이름을 지정해서 파일을 가져온다.
- 컴퓨터 저장장치 내의 파일 위치는 **경로** `path`라고 함

2. 사용 목적에 따라 사용

- 내용 확인 및 새로운 내용 추가 또는 기존 내용 삭제 가능

3. 파일 닫기

- 파일을 모두 사용하고 나면 이 메모리를 시스템에서 사용할 수 있도록 반환



[그림 9-2] 파일을 열고 사용한 후 닫는 3단계의 절차



9.7 파일 입출력



9.7.2 파일 쓰기과 모드

- `open()` 함수는 'hello.txt'라는 파일을 열어서 가져오는 역할
 - 'w'라는 파일 오픈 모드를 지정하여 가져옴 (쓰기 전용)
- `open()` 함수는 파일 객체를 반환
- 파일 객체 `f`는 `write()` 메소드를 이용하여 지정된 형식으로 파일에 지정된 텍스트를 쓰는 일을 한다.
- `f.close()` 메소드는 파일을 닫아줌



코드 9-10: 파일 열기와 쓰기, 파일 닫기의 표현

`file_write_hello.py`

```
f = open('hello.txt', 'w')    # 1) 파일을 연다.  
f.write('Hello World!')      # 2) hello.txt 파일에 문자를 쓴다.  
f.close()                   # 3) 파일을 닫는다.
```



9.7 파일 입출력



open() 함수 [표 9-3] 파일 오픈을 위한 여러 가지 모드와 그 의미

모드	의미
r	Read의 약자로 파일을 읽기 전용 모드로 연다. 이 모드는 파일 열기의 기본 모드이다. - 파일이 없을 경우 에러가 발생한다. - 읽기 전용 모드이므로 파일을 쓰려고 하면 오류가 발생한다.
w	Write의 약자로 파일을 쓰기 전용 모드로 연다. - 파일이 없을 경우 파일을 생성하며, 파일이 있을 경우 덮어쓰게 된다.
a	Append의 약자로 파일을 쓰기 모드로 열어서 기존 파일의 뒤에 새로 작성된 내용을 추가한다. - 기존 파일이 없을 경우 새로 파일을 만들어서 추가한다.
x	이 모드는 쓰기 전용으로 파일을 여는데, 파일을 새로 만들기 위해서 사용한다. - 이 모드는 a나 w와 달리 파일이 있을 경우 에러가 발생한다.
+	+ 기호는 읽기 쓰기 모드로 'r+'나 'w+', 'r+t'와 같은 조합으로 사용할 수 있다. 'r+'와 같이 표기하면 읽기 모드로 열어서 쓰기까지 가능하다. 반면, 'w+'와 같이 표기하면 쓰기 모드로 열어서 읽기까지 가능하다.



9.7 파일 입출력



이진 파일을 다룰지, 텍스트 파일을 다룰지를 지정하는 모드

[표 9-4] 텍스트 파일과 이진 파일을 위한 모드의 종류

파일 모드	의미
t	Text의 약자로 텍스트 파일 형식으로 파일을 열거나 생성한다. 이 모드는 파일 열기의 기본 모드이다.
b	Binary의 약자로 이진 파일 형식으로 파일을 열거나 생성한다.



9.7 파일 입출력



이진 파일을 다룰지 텍스트 파일을 다룰지를 지정하는 모드

```
f = open('hello.txt', 'w') # 파일 열기
```

파일을 쓰기모드(w)로 읽어서 파일 객체를 반환 : write() 메소드 호출하여 지정된 파일에 텍스트를 쓸 수 있음

```
f = open('hello.txt', 'wt') # 파일을 텍스트로 열기
```

위의 코드와 동일함 텍스트(t) 모드로 파일을 열어서 쓸 수 있음

```
f = open('hello.txt', 'wb') # 파일을 바이너리로 열기
```

wb모드는 바이너리 모드로 파일 쓰기 가능

```
f = open('hello.txt', 'r+t') # 텍스트 읽기 쓰기 모드
```

기본모드는 읽기 모드이지만 쓰기도 가능, 파일 없으면 오류 발생

```
f = open('hello.txt', 'w+t') # 텍스트 쓰기 읽기 모드
```

기본모드는 쓰기 모드이지만 읽기도 가능함

```
f = open('hello.txt', 'a+t') # 텍스트 추가 모드
```

파일의 모든 내용을 남겨두고 맨 뒤에 추가함. 읽기도 가능하며 파일이 없을 경우 만든다.

```
f.write('Hello World!') # hello.txt 파일에 이어쓰기
```



9.7 파일 입출력



파일 닫기

- ▣ 파일은 스스로 닫는 방법을 알고 있음
 - 파일 객체의 `close()` 함수 사용
 - 열린 파일의 이름표를 사용

```
파일_변수.close()      # 파일을 닫음
```

- 예

```
f.close()
```




9.7 파일 입출력



9.7.3 파일 읽기

- read() 메소드를 통해서 파일 읽어오기
- 파일의 모든 문자열(전체 내용)은 줄 바꿈에 관계없이 하나의 문자열로 반환됨



코드 9-11: 파일 열기와 읽기, 파일 닫기의 표현
file_read_hello.py

```
f = open('hello.txt', 'r') # 파일을 연다.  
s = f.read()              # hello.txt 파일을 읽는다.  
print(s)                  # 파일의 내용을 출력한다.  
f.close()                 # 파일을 닫는다.
```

hello.txt 파일을 읽어서 print() 함수로 출력하는 프로그램

실행결과

Hello World!



9.7 파일 입출력



9.7.3 파일 읽기

- `f.read()` 메소드의 인자로 5를 주면 파일로부터 5개의 문자를 읽어 와서 이 문자를 문자열로 반환
- 'Hello'의 다섯 문자만 출력된다.



코드 9-12: `read()` 메소드를 이용하여 지정된 문자 크기만큼 읽기
`file_read_5char.py`

```
f = open('hello.txt', 'r')    # 파일을 연다.  
s = f.read(5)                # hello.txt 파일의 다섯 문자를 읽는다.  
print(s)                     # 파일의 내용을 출력한다.  
f.close()                    # 파일을 닫는다.
```

실행결과

Hello



9.7 파일 입출력



줄 단위로 파일 읽기

▣ readline() 함수 쓰기

- 한 줄(line)의 텍스트를 읽어서 반환
- 파일에 끝에 도달하면 False (빈 문자열) 가 됨
- 주로 while 문과 함께 쓰임

```
f = open("t.txt")      # f = open("t.txt", "r")
line = f.readline()    # 한 줄 읽어옴
while line:            # 파일의 끝에 도달하면 False
    print(line, end = '')
    line = f.readline()
f.close()              # 파일 닫기
```



파일 읽기

- ▣ 파일 내용을 리스트로 구성
- ▣ `readlines()` 함수 활용
 - 파일 전체를 읽어서 리스트에 저장

```
f = open("t.txt") # f = open("t.txt", 'r')
# 파일 전체의 내용을 리스트로 만듦
lines = f.readlines()
for line in lines:
    print(line, end = '')
f.close()          # 파일 닫기
```



9.7 파일 입출력



9.7.4 파일 추가하기

- 'a+' 모드를 사용
 - 이미 만들어진 파일의 뒤쪽에 새로운 내용을 추가
- a는 추가(append) 모드
- +는 기존 파일이 없을 경우 새롭게 파일을 생성하는 모드



9.7 파일 입출력



9.7.5 파일 쓰기과 읽기 예제

- 파일로부터 정수를 읽어서 이 정수들의 합과 평균을 출력
- 파일에 저장된 10은 문자열 '10'이므로 int()를 이용하여 정수로 바꾸어야 덧셈 연산이 가능



코드 9-17: 파일로부터 정수를 입력 받아 합과 평균을 계산
file_read_5num.py

```
f = open('data5.txt','r') # 읽기 모드로 파일 열기
su = 0
for _ in range(5):
    n = int(f.readline()) # data5.txt 파일의 한 줄 내의 숫자를 읽어서
    su += n # su에 누적해서 더한다.
print(f'다섯 숫자의 합 = {su}, 평균 = {su/5}') # 합과 평균 출력
f.close()
```

실행결과

다섯 숫자의 합 = 150, 평균 = 30.0



9.8 with 문법



- with ~ as : 문을 사용하여 간단하게 사용



코드 9-22: 파일 열기와 쓰기, 파일 닫기의 표현 3
file_write_hello3.py

```
with open('hello.txt', 'w') as f: # 파일 열기와 닫기를 자동 수행함
    f.write('Hello World!')        # hello.txt 파일에 쓰기
```

- 파일 쓰기가 완료되면 자동으로 f.close()가 수행되기 때문에 finally 절을 사용할 필요가 없음

실습문제 1

□ 문제

- "Hello Python"이라는 문자열을 파일로 저장하기 위하여 my_hello.txt라는 파일을 열고 쓰기를 시도하였다. 이때 이 파일이 이미 존재하고 있으며, 다음 그림과 같이 읽기 전용으로 설정되어 있다고 가정하자.

- (1) 이 상황에서는 어떤 예외가 발생하는가?
- (2) 실제로 파일을 만들어서 이러한 상황을 테스트하는 프로그램을 작성하여 예외를 출력하여라.
- (3) 이 예외를 처리하는 try-exception 문을 구현하여라.

•Windows:

- 1.Right-click my_hello.txt
- 2.Choose **Properties**
- 3.Under **Attributes**, check **Read-only**
- 4.Click **Apply** and **OK**

•macOS / Linux (terminal):

```
chmod 444 my_hello.txt
```

□ 실행 결과 예시

```
Error: [Errno 13] Permission denied: 'C:\\Temp\\Py_examples\\my_hello.txt'
```


실습문제 2

□ 문제

- 다음과 같은 내용을 포함하고 있는 hello.txt 파일을 텍스트 편집기를 이용하여 작성하시오.

```
hello.txt 파일
Hello.
Hi, there.
```

- ① 이제 이 파일을 open() 함수를 통해 읽어서 이 파일의 내용을 화면에 출력하시오.
- ② 이 파일을 open() 함수를 통해 읽어서 원래 파일의 내용 아래쪽에 'Welcome to Python!.'을 추가한 후 저장하시오. 그리고 다시 한 번 파일을 읽어서 다음과 같이 화면에 출력하시오.

□ 실행 결과 예시

1

```
hello.txt file:
Hello.
Hi, there.
```

2

```
hello.txt file:
Hello.
Hi, there.
Welcome to Python!
```

실습문제 3

□ 문제

- ▣ 사용자로부터 파일 이름을 입력받고 내용을 화면에 출력하는 프로그램 작성

□ 요구사항

- ▣ 작업하고 있는 디렉토리 'chosun2025.txt' 파일 생성하시오.
- ▣ 사용자가 입력한 파일이 존재하지 않으면 한 번 더 입력할 수 있도록 함
- ▣ 두 번째 입력한 파일은 있다고 가정
- ▣ 파일이 없는지 확인하는 것은 예외처리를 사용

□ 실행 결과 예시

```
Enter the file name: chosun2025
There is not such file. Enter again the file name: chosun2025.txt

File content:
Hello!

Chosun 2025-2!!!
```

실습문제 4

□ 문제

- ▣ 텍스트 파일 두 개를 병합하는 프로그램을 작성. 두 번째 파일의 내용을 첫 번째 파일의 내용에 붙여 새로운 파일에 저장

□ 요구사항

- ▣ 사용자로부터 세 개 파일 이름을 입력 받기
- ▣ 입력 받은 첫 번째와 두 번째 파일 이름은 존재하는 텍스트 파일의 이름이라고 가정
- ▣ 첫 번째 파일과 두 번째 파일을 붙여서 세 번째 파일에 저장
- ▣ With 문 사용해야 됨

□ 실행 예시

```
Enter the name of the first file to link: my_hello.txt
Enter the name of the second file to link: chosun2025.txt
Enter the name of the file to save: merged.txt

The merged file content is:
wellcome to python!
I love python!!!Hello!

Chosun 2025-2!!!
```

실습문제 4

□ 문제

- 파일에서 문자열을 검색하는 프로그램 작성
- 검색된 줄을 다음 형태로 화면에 출력

줄 번호: 검색할_단어가_있는_문자열

- 검색 단어가 파일에 없으면 "검색 단어가 파일에 없습니다"라고 출력

□ 요구사항

- 파일에서 검색할 문자열과 검색할 파일 이름을 사용자로 부터 입력 받기
- 사용자가 입력한 파일은 반드시 존재한다고 가정
- 줄 번호는 1부터 시작

□ 실행 결과

```
Enter the file name: merged.txt
Enter the word to search for: person
That word is not in the file
```

```
Enter the file name: merged.txt
Enter the word to search for: python
1:welcome to python!
2:I love python!!!Hello!
```

실습문제 5

- 문제: 다음과 같은 내용을 포함하고 있는 number1to10.txt 파일을 한 줄씩 읽으라.
 - ▣ (1) 이 파일을 읽어서 각 행의 정수에 곱하기 10을 수행한 후 다음과 같은 numberby10.txt라는 이름의 파일로 저장하여라.

number1to10.txt 파일
1
2
3
4
5
6
7
8
9
10

numberby10.txt 파일
10
20
30
40
50
60
70
80
90
100

- ▣ (2) 위의 두 파일을 읽어서 다음과 같이 각 행의 값을 하나의 내용으로 묶어서 저장하는 프로그램을 작성하여라. 이 파일의 이름은 merge.txt이다.

merge.txt 파일
1 : 10
2 : 20
3 : 30
4 : 40
5 : 50
6 : 60
7 : 70
8 : 80
9 : 90
10 : 100

실습문제 6

□ 문제: 다음과 같은 기능을 수행하는 파이썬 프로그램을 작성하여라

- ① random 모듈의 randint() 메소드를 사용하여 1에서 1,000까지의 임의의 정수를 10개 생성하여 random_numbers.txt라는 이름의 파일로 저장하여라.

random_numbers.txt 파일

67 781 400 869 370 401 245 418 185 804

- ② random_numbers.txt라는 파일로부터 숫자를 읽어 들인 후 이들 중 짝수 값을 가지는 정수를 저장하는 random_even.txt라는 파일을 생성하여라.

random_even.txt 파일

400 370 418 804

- ③ random_numbers.txt라는 파일로부터 숫자를 읽어 들인 후 이들 중 짝수 값을 가지는 정수를 저장하는 random_even.txt라는 파일을 생성하여라.

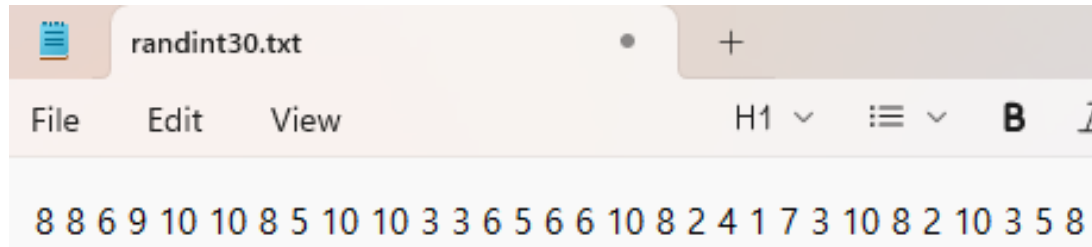
randint30.txt 파일

1 7 7 5 9 7 10 5 9 4 7 2 5 2 10 8 9 5 5 6 10 1 3 1 9 10 7 3 7 2

실습문제 7

□ 문제

- ① 1에서 10 사이의 랜덤한 숫자를 30개 가진 randint30.txt라는 파일을 random 모듈을 사용하여 생성하시오. 이 때 생성한 숫자는 다음과 같이 공백으로 구분하여 나열하여라.



- ② 위의 randint30.txt라는 파일을 읽어서 1에서 10까지 정수의 출현 횟수를 다음과 같이 출력하여라.

□ 실행 예시:

```
[8, 8, 6, 9, 10, 10, 8, 5, 10, 10, 3, 3, 6, 5, 6, 6, 10, 8, 2, 4, 1, 7, 3, 10, 8, 2, 10, 3, 5, 8]
1 : 1 elements
2 : 2 elements
3 : 4 elements
4 : 1 elements
5 : 3 elements
6 : 4 elements
7 : 1 elements
8 : 6 elements
9 : 1 elements
10 : 7 elements
```